

generated Makefiles. -O0 disables optimisation. The user can explicitly specify the level of optimisation with -On, where n is an integer. The highest level of optimisation is -O3.

Verilator can also produce SystemC output with -sc. SystemVerilog support also exists, and the relevant code can be generated with -sv.

If you want to analyse the intermediate steps in the compilation process, you can use --dump-tree:

```
$ verilator --dump-tree --cc hello.v --exe main.cpp

dot -Tps -o ~/a.ps obj_dir/Vhello_01_linkcells.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_21_task_call.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_33_gate_simp.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_34_gate_opt.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_40_orderq_pre.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_41_orderq_acyc.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_42_orderq_order.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_43_orderq_domain.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_44_ordermv_simpl.dot
dot -Tps -o ~/a.ps obj_dir/Vhello_45_orderq_done.dot
```

--cdc performs a clock domain crossing (CDC) analysis, which can be invoked on an input module as follows:

```
$ verilator -cc input.v --cdc
```

Verilator can check for warnings. Consider the following Verilog code:

```
module test;

    wire a, b, c;
    and (x, b, c);

endmodule
```

An implicit warning is produced by Verilator as shown below:

```
$ verilator -cc lint.v --exe main.cpp

%Warning-IMPLICIT: lint.v:4: Signal definition not found,
creating implicitly: x
%Warning-IMPLICIT: Use /* verilator lint_off IMPLICIT
*/ and lint_on around source to disable this message.
%Error: Exiting due to 1 warning(s)
%Error: Command Failed verilator_bin -cc lint.v --exe main.cpp
```

Lint warnings can be disabled with -Wno-lint.

```
$ verilator -cc lint.v --exe main.cpp -Wno-lint
$
```

Verilator can also produce a useful statistics file with --stats. Vhello_stats.txt file is created in objdir for hello.v module.

```
$ verilator --cc hello.v --exe main.cpp --stats
```

There also exists --profile-cfuncs that adds profiling code to the generated C++ files. Tools like gprof [2] can be used on the generated output to analyse the input Verilog code.

```
$ verilator -cc hello.v --exe main.cpp --profile-cfuncs
```

The following is a half-adder example:

```
module ha(a, b, sum, carry);

    input a;
    input b;

    output sum;
    output carry;

    assign carry = a & b;
    assign sum = a ^ b;

endmodule
```

The simulation to test the half-adder example is given by main.cpp file.

```
#include "Vhalfadder.h"
#include <verilated.h>
#include "verilated_vcd_c.h"

unsigned int main_time = 0;

double sc_time_stamp () {
    return main_time;
}

int
main (int argc, char **argv, char **env)
{
    Verilated::commandArgs(argc, argv);

    Vhalfadder* top = new Vhalfadder;

    Verilated::traceEverOn(true);
    VerilatedVcdC* tfp = new VerilatedVcdC;

    top->trace (tfp, 99);
    tfp->open ("counter.vcd");

    top->sum    = 0;
    top->carry  = 0;

    top->a = 0;
    top->b = 0;

    while (main_time < 5 && !Verilated::gotFinish()) {

        if ((main_time % 4) == 0) {
            top->a = 0;
            top->b = 0;
        }

        if ((main_time % 4) == 1) {
            top->a = 1;
            top->b = 0;
        }
    }
}
```